Name : Shivraj Vaibhav Jagtap
Roll No : EN24107060
Class : SY - AI \& DS
Subject : Python for Visual AI

Assignment 9:

An organisation wants to automate daily reports from CSV files. Automate data loading & cleaning, Generate summary reports, Save output as CSV / PDF, and schedule tasks using Python (cron / task scheduler)

Required Libraries

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
```

Loading Dataset

In [ ]:
```
df = pd.read_csv("/content/sales_data_sample.csv", encoding="latin1")
df.head()
```

Out[ ]:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	ORDERDATE	ST
0	10107	30	95.70	2	2871.00	2/24/2003 0:00	Sh
1	10121	34	81.35	5	2765.90	5/7/2003 0:00	Sh
2	10134	41	94.74	2	3884.34	7/1/2003 0:00	Sh
3	10145	45	83.26	6	3746.70	8/25/2003 0:00	Sh
4	10159	49	100.00	14	5205.27	10/10/2003 0:00	Sh

5 rows × 25 columns

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Data Cleaning/Preprocessing

In [ ]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   ORDERNUMBER       2823 non-null   int64  
 1   QUANTITYORDERED   2823 non-null   int64  
 2   PRICEEACH         2823 non-null   float64
 3   ORDERLINENUMBER   2823 non-null   int64  
 4   SALES             2823 non-null   float64
 5   ORDERDATE         2823 non-null   object 
 6   STATUS            2823 non-null   object 
 7   QTR_ID            2823 non-null   int64  
 8   MONTH_ID          2823 non-null   int64  
 9   YEAR_ID           2823 non-null   int64  
 10  PRODUCTLINE       2823 non-null   object 
 11  MSRP              2823 non-null   int64  
 12  PRODUCTCODE       2823 non-null   object 
 13  CUSTOMERNAME      2823 non-null   object 
 14  PHONE             2823 non-null   object 
 15  ADDRESSLINE1      2823 non-null   object 
 16  ADDRESSLINE2      302 non-null    object 
 17  CITY              2823 non-null   object 
 18  STATE             1337 non-null   object 
 19  POSTALCODE        2747 non-null   object 
 20  COUNTRY           2823 non-null   object 
 21  TERRITORY         1749 non-null   object 
 22  CONTACTLASTNAME   2823 non-null   object 
 23  CONTACTFIRSTNAME  2823 non-null   object 
 24  DEALSIZE          2823 non-null   object 
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB
```

In [ ]:

```
df.shape
```

Out[ ]:

```
(2823, 25)
```

In [ ]:

```
df.isnull().sum()
```

Out[ ]:

	0
ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	0
SALES	0
ORDERDATE	0
STATUS	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	2521
CITY	0
STATE	1486
POSTALCODE	76
COUNTRY	0
TERRITORY	1074
CONTACTLASTNAME	0
CONTACTFIRSTNAME	0
DEALSIZE	0

dtype: int64

In [ ]:

```
df.drop_duplicates(inplace=True)
```

In [ ]:

```
df.fillna(0, inplace=True)
```

In [ ]:

```
df.isnull().sum()
```

Out[ ]:

	0
ORDERNUMBER	0
QUANTITYORDERED	0
PRICEEACH	0
ORDERLINENUMBER	0

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	0
SALES	0
ORDERDATE	0
STATUS	0
QTR_ID	0
MONTH_ID	0
YEAR_ID	0
PRODUCTLINE	0
MSRP	0
PRODUCTCODE	0
CUSTOMERNAME	0
PHONE	0
ADDRESSLINE1	0
ADDRESSLINE2	0
CITY	0
STATE	0
POSTALCODE	0
COUNTRY	0
TERRITORY	0
CONTACTLASTNAME	0
CONTACTFIRSTNAME	0
DEALSIZE	0

dtype: int64

In [ ]:

```
df["ORDERDATE"] = pd.to_datetime(df["ORDERDATE"], errors="coerce")
```

Total Sales :

$$Total\ Sales = \sum_{i=1}^{n} \text{Sales}_i$$

Total Orders :

$$TotalOrders = n$$

In [ ]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
total_sales = df["SALES"].sum()
total_orders = df.shape[0]
```

Estimated Profit

$$\text{Total Profit} = \text{Total Sales} \times r$$

where r is the profit ratio (e.g., 0.10).

In [ ]:

```
total_profit = total_sales * 0.10
```

Sales by Country

$$\text{Sales}_{country} = \sum \text{Sales grouped by country}$$

In [ ]:

```
country_sales = df.groupby("COUNTRY")["SALES"].sum()
```

Graphical Representation

Bar chart plotted using matplotlib to compare country-wise sales distribution

In [ ]:

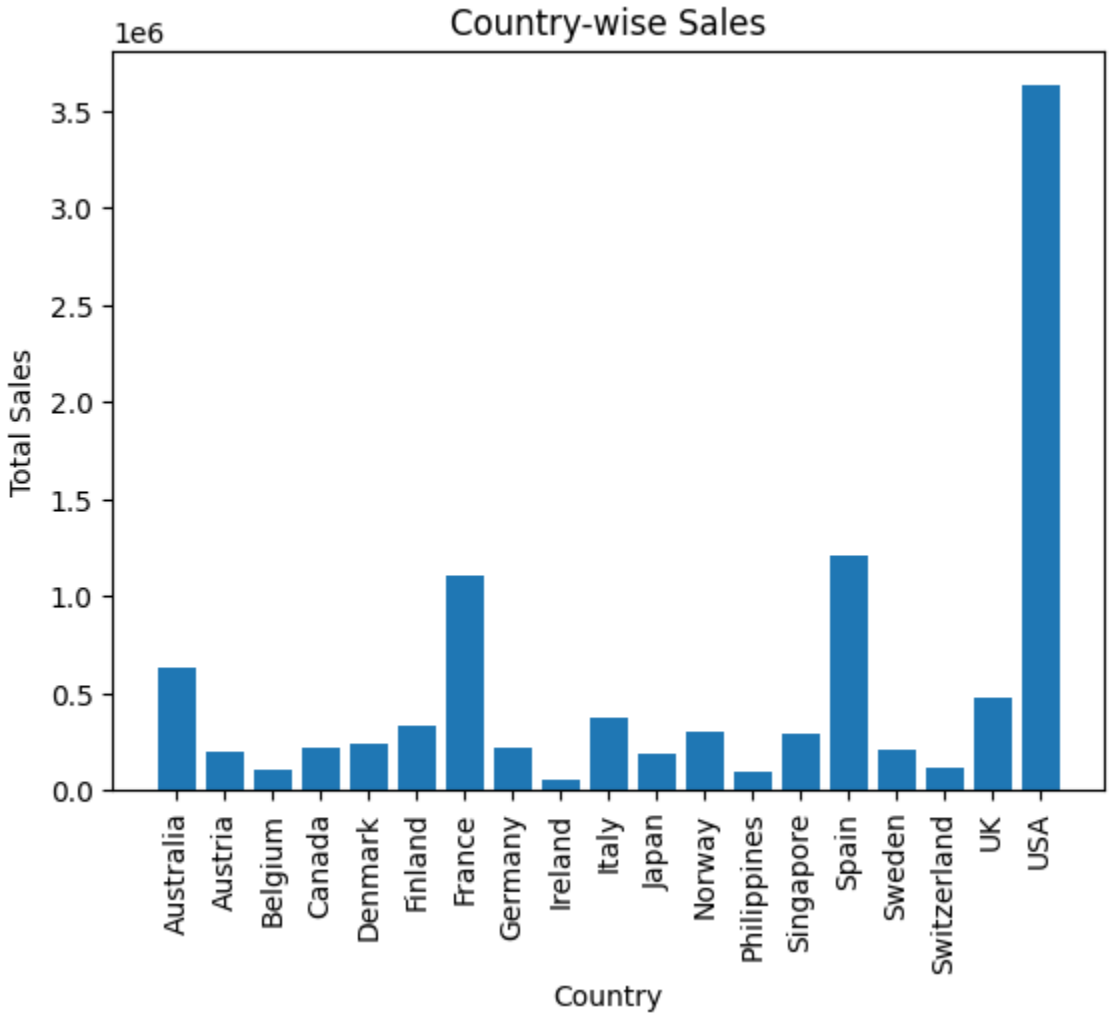
```
plt.figure()
plt.bar(country_sales.index, country_sales.values)

plt.xlabel("Country")
plt.ylabel("Total Sales")
plt.title("Country-wise Sales")

plt.xticks(rotation=90)

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Summary Calculation

A dictionary is created to store key performance metrics of the dataset:

$$Total\ Orders : Number\ of\ records\ in\ the\ dataset$$
$$Total\ Sales : Sum\ of\ all\ sales\ values, rounded\ to\ 2\ decimal\ places$$
$$Total\ Profit(Estimated) : Calculated\ profit\ based\ on\ total\ sales$$

Summary :

$$\begin{cases} \text{Total Orders} = n \\ \text{Total Sales} = \text{round}\left(\sum \text{Sales}, 2\right) \\ \text{Total Profit} = \text{round}(\text{Total Sales} \times r, 2) \end{cases}$$

The summary is then displayed using the print function.

In [ ]:

```
summary = {
    "Total Orders": total_orders,
    "Total Sales": round(total_sales, 2),
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    "Total Profit (Estimated)": round(total_profit, 2)
}

print(summary)
```

```
{'Total Orders': 2823, 'Total Sales': np.float64(10032628.85), 'Total Profit (Estimate
d)': np.float64(1003262.88)}
```

CSV Report Generation

1. Get current date and time
2. Convert summary dictionary into DataFrame
3. Create CSV file name
4. Save DataFrame as CSV file
5. Display confirmation message

In [ ]:

```
date = datetime.now().strftime("%Y%m%d_%H%M")

summary_df = pd.DataFrame(list(summary.items()), columns=["Metric", "Value"])

csv_name = f"sales_summary_{date}.csv"
summary_df.to_csv(csv_name, index=False)

print("CSV report saved")
```

```
CSV report saved
```

PDF Report Generation

1. Initialize PDF and add a new page using FPDF
2. Set title 'Daily Sales Report'
3. Print summary values (Total Orders, Sales, Profit)
4. Display country-wise sales data
5. Create PDF file name
6. Save PDF file

pdf.output(pdf_name)

In [ ]:

```
pdf = FPDF()
pdf.add_page()

pdf.set_font("Arial", size=14)
pdf.cell(200, 10, "Daily Sales Report", ln=True)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
pdf.set_font("Arial", size=12)

for key, value in summary.items():
    pdf.cell(200, 10, f"{key}: {value}", ln=True)

pdf.cell(200, 10, "", ln=True)

pdf.cell(200, 10, "Sales by Country", ln=True)

for country, sales in country_sales.items():
    pdf.cell(200, 10, f"{country}: {round(sales, 2)}", ln=True)


pdf_name = f"sales_report_{date}.pdf"
pdf.output(pdf_name)

print("PDF report saved")
```

```
PDF report saved
```

Conclusion

The experiment successfully automated daily report generation from CSV data using Python. Data loading, cleaning, and summary analysis were performed efficiently without manual effort. Reports were generated and saved in both CSV and PDF formats for easy access and sharing. The use of task scheduling (cron/Task Scheduler) ensures the process runs automatically on a daily basis.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF